

MANUAL TÉCNICO · PROYECTO DE AULA

Documento de Despliegue Técnico y Arquitectura del Sistema JAS

Panel de Control Financiero y Contable Multiempresa

PROYECTO	Proyecto de Aula – Manual Técnico de Despliegue de Infraestructura Local
EQUIPO	Equipo de Desarrollo e Implementación del Sistema JAS
ÁREA	Área de Ingeniería de Software y Arquitectura de Sistemas
DOCENTE ORIENTADOR	Docente Orientador del Proyecto
FECHA	3 de noviembre de 2026

TECNOLOGÍAS DEL SISTEMA

.NET 8 | PostgreSQL | JWT | Swagger | Chart.js

Resumen

El presente documento constituye la guía técnica independiente de arquitectura y procedimiento de despliegue para el sistema JAS – Panel de Control Financiero y Contable, una solución informática tipo DSS (Decision Support System) orientada a la gestión multiempresa en Colombia. El sistema está estructurado mediante una arquitectura desacoplada de tres capas compuesta por una API REST construida en ASP.NET Core 8, una base de datos relacional PostgreSQL (escuchando en el puerto 5433) y una interfaz de usuario cliente desarrollada en JavaScript estático (vanilla HTML/CSS/JS) con integración de bibliotecas analíticas mediante CDN. Se documentan de manera exhaustiva los parámetros de configuración en appsettings, la cadena de conexión por defecto, las reglas de acceso por empresa (AuthorizeEmpresaAccessFilter), la política de seguridad CORS abierta (AllowFrontend) y la autenticación mediante tokens JWT y hash de contraseñas. Adicionalmente, se realiza un diagnóstico crítico sobre la ejecución automática de scripts SQL en Program.cs, identificando una falla en la ruta relativa del archivo migration_update.sql, y se provee la metodología estandarizada para el despliegue local manual y las recomendaciones de preparación para entornos de producción.

Palabras clave: ASP.NET Core 8, PostgreSQL, API REST, arquitectura multiempresa, despliegue local, JWT, Entity Framework Core.

Introducción y Alcance del Sistema

En el desarrollo de soluciones de software empresarial orientadas a la toma de decisiones financieras (DSS multiempresa), la correcta configuración de la infraestructura de despliegue y la definición clara de la arquitectura del sistema resultan determinantes para garantizar la disponibilidad, integridad y escalabilidad de los servicios. El proyecto JAS – Panel de Control Financiero y Contable ha sido diseñado para operar en el contexto corporativo colombiano, permitiendo la consolidación y gestión paramétrica de información contable, terceros, centros de costos, transacciones, presupuestos e indicadores de rendimiento económico.

El alcance del presente documento abarca la especificación del estado actual del código entregado del sistema, detallando su ejecución bajo un esquema de despliegue manual en entorno de desarrollo local. Dado que la entrega actual no incluye archivos de orquestación (Dockerfile, docker-compose.yml), tuberías de integración y despliegue continuo (CI/CD) ni archivo de solución contenedor (.sln), este manual establece el procedimiento paso a paso para la puesta en marcha coordinada de los servicios del backend, el motor de base de datos y la interfaz cliente estática de manera autónoma e independiente.

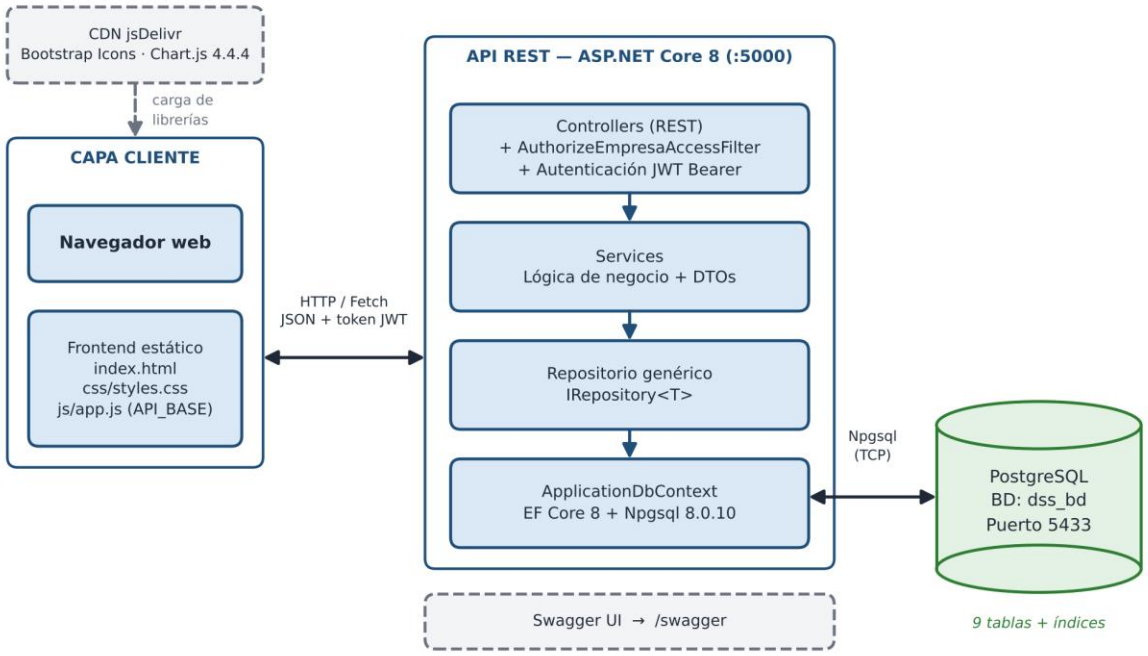
Arquitectura Técnica y Componentes del Sistema

El sistema JAS se fundamenta en un patrón arquitectónico por capas claramente segregadas (Controllers -> Services -> Generic Repository IRepository -> Entities / ApplicationDbContext), interactuando mediante objetos de transferencia de datos (DTOs) para las operaciones de entrada y salida a través de la red HTTP. El flujo de información inicia cuando el navegador cliente ejecuta la interfaz estática y realiza peticiones asincrónicas mediante la API Fetch hacia los endpoints expuestos por el backend.

La Figura 1 sintetiza esta arquitectura, mostrando la relación entre el navegador, la interfaz estática, la API REST y el motor de base de datos PostgreSQL.

Figura 1

Arquitectura por Capas del Sistema JAS



Nota. Elaboración propia a partir del análisis del código fuente del proyecto.

Tabla 1

Matriz de Componentes y Tecnologías de la Arquitectura JAS

Componente	Tecnología Base	Ubicación en el Proyecto	Puerto / URL por Defecto
API REST Backend	ASP.NET Core Web API (.NET 8 net8.0)	Backend/DssMultiEmpresa.API	Puerto Local 5000
Base de Datos Relacional	PostgreSQL (EF Core 8 + Npgsql 8.0.10)	Database/init.sql, Database/migration_update.sql	Puerto 5433 (BD dss_bd)
Frontend Cliente	HTML5, CSS3, JS Vanilla; Bootstrap Icons, Chart.js 4.4.4	Frontend/	Cliente estático (index.html)
Módulo Autenticación	JWT Bearer + Hash de Contraseñas	Program.cs, Services/AuthService.cs	Rutas Auth Login y Register

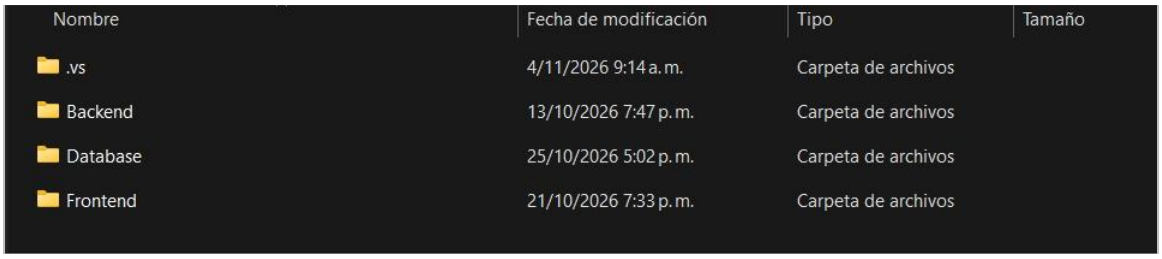
Componente	Tecnología Base	Ubicación en el Proyecto	Puerto / URL por Defecto
Documentación de API	Swagger UI (Swashbuckle 10.3.0)	Program.cs	Ruta /swagger

Nota. Especificaciones del entorno de componentes del sistema JAS en despliegue local.

La estructura física del repositorio entregado se organiza en tres carpetas principales (Backend, Database y Frontend), tal como se observa en la Figura 2.

Figura 2

Estructura de Carpetas del Proyecto ProyectoDeAula



Nota. Captura del explorador de archivos del equipo de desarrollo. La carpeta .vs corresponde a archivos locales de Visual Studio y no forma parte de la solución.

API REST Backend (.NET 8)

El núcleo de procesamientos del sistema reside en la aplicación ASP.NET Core Web API (net8.0). La API implementa la lógica de negocio a través de controladores y servicios especializados, integrando un middleware de validación JWT que restringe el acceso a los recursos según el identificador de la empresa activa (AuthorizeEmpresaAccessFilter). El acceso a datos se gestiona mediante Entity Framework Core 8 combinado con el proveedor Npgsql 8.0.10 para PostgreSQL.

Base de Datos Relacional (PostgreSQL)

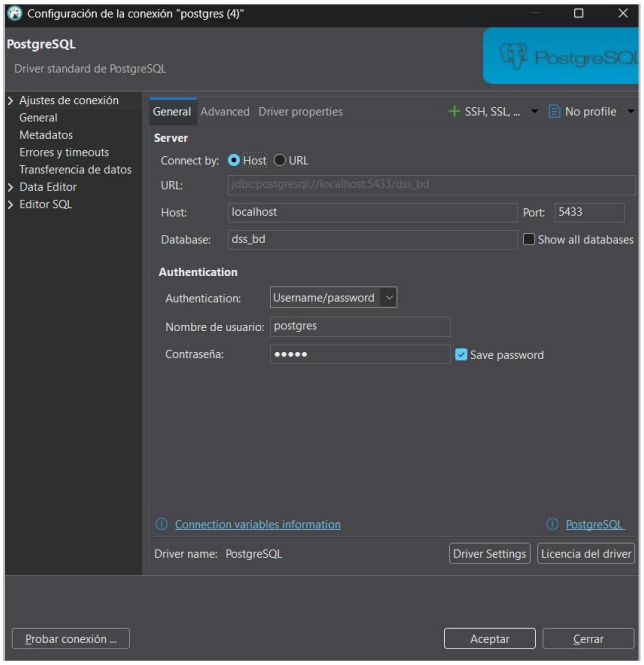
El motor de almacenamiento corresponde a una instancia relacional de PostgreSQL configurada explícitamente en el puerto host 5433 (diferenciándose del puerto predeterminado 5432). La estructura lógica se compone de la base de datos dss_bd y nueve tablas principales:

usuarios, empresas, categorias_contables, cuentas_contables, terceros, centros_costo, transacciones_contables, presupuestos e indicadores_dss, complementadas con sus respectivos índices relacionales.

En la Figura 3 se presentan los parámetros de conexión configurados en el cliente de base de datos (host localhost, puerto 5433 y base de datos dss_bd), los cuales coinciden con la cadena de conexión definida en la API.

Figura 3

Parámetros de Conexión a la Base de Datos dss_bd



Nota. Captura de pantalla de la configuración de conexión en DBeaver. La contraseña permanece oculta por el propio cliente.

Frontend Estático y Consumo de Servicios

La capa cliente ha sido implementada mediante tecnologías web estándar (HTML5, CSS3 y JavaScript nativo en Frontend/) sin dependencia de frameworks de renderizado en el servidor. La aplicación consume la API mediante peticiones HTTP asincrónicas configuradas mediante la

constante global API_BASE en el archivo Frontend/js/app.js (línea 6). Las visualizaciones gráficas de tableros financieros emplean la librería Chart.js 4.4.4 importada vía CDN.

Requisitos del Entorno y Configuración

Para la ejecución satisfactoria del entorno local de desarrollo se requiere contar con la instalación previa de las siguientes herramientas de software:

• .NET SDK 8.0 (requerido para compilación, restauración de paquetes NuGet y ejecución) o ASP.NET Core Runtime 8.0 si se despliega sobre binarios previamente publicados.

• Motor de PostgreSQL activo escuchando en el puerto 5433.

• Navegador web moderno con conectividad a internet para la carga de componentes visuales (Bootstrap Icons y Chart.js desde CDN).

• Cliente de gestión SQL opcional (psql, pgAdmin 4 o DBeaver) para la ejecución manual de scripts de base de datos.

Cadena de Conexión y Parámetros de Configuración

La configuración del servidor backend se administra mediante el archivo de propiedades appsettings.json, cuyos parámetros clave se detallan a continuación en la Tabla 2.

Tabla 2

Parámetros de Configuración del Sistema en appsettings.json

Clave de Configuración	Valor Asignado por Defecto	Propósito y Función en la Aplicación
DefaultConnection	Host=localhost; Port=5433; Database=dss_bd; Username=postgres;	Establece los parámetros de autenticación y red para la conexión con PostgreSQL.
JwtSettings SecretKey	Clave alfanumérica de firma JWT en appsettings.json	Firma criptográfica para la emisión y verificación de tokens JWT.

Clave de Configuración	Valor Asignado por Defecto	Propósito y Función en la Aplicación
JwtSettings Issuer / Audience	DssApi / DssFrontend	Identificadores de emisor y audiencia válidos para el esquema de autenticación.
AllowedHosts	*	Define los nombres de dominio habilitados para consumir el backend.
Logging LogLevel	Information (ASP.NET en Warning)	Controla la verbosidad de los registros de depuración y eventos en consola.

Nota. Valores extraídos directamente de la configuración appsettings.json entregada en la solución.

Políticas de Seguridad y Configuración CORS

El perfil de lanzamiento de la API se especifica en Backend/DssMultiEmpresa.API/Properties/launchSettings.json bajo el protocolo HTTP en el puerto 5000 con la variable de entorno ASPNETCORE_ENVIRONMENT=Development. Para mitigar el bloqueo por políticas de origen cruzado en los navegadores al abrir el cliente estático desde rutas de archivo locales o servidores independientes, el servicio habilita la política CORS denominada AllowFrontend con comodines abiertos (AllowAnyOrigin, AllowAnyMethod, AllowAnyHeader).

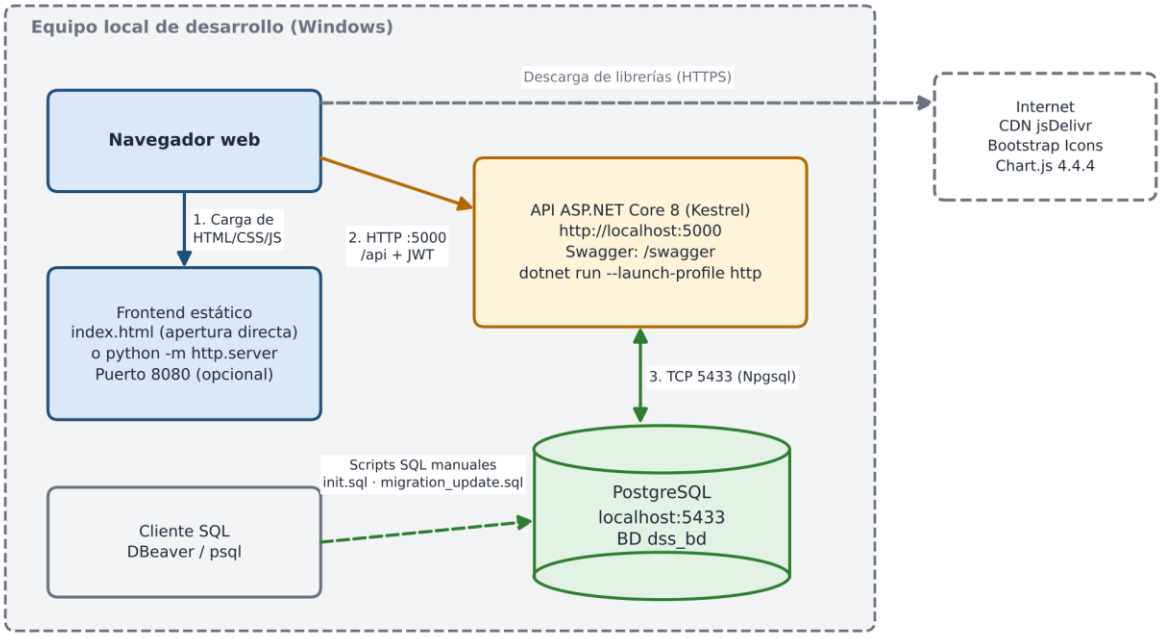
Procedimiento Paso a Paso para el Despliegue Local

El despliegue operativo del sistema en un entorno de desarrollo local exige la ejecución secuencial de tres fases obligatorias: inicialización de la base de datos, puesta en marcha del servidor API y publicación de la interfaz cliente.

La Figura 4 ilustra la topología del entorno local, indicando los puertos utilizados por cada componente y el sentido de las comunicaciones.

Figura 4

Diagrama de Despliegue en el Entorno Local

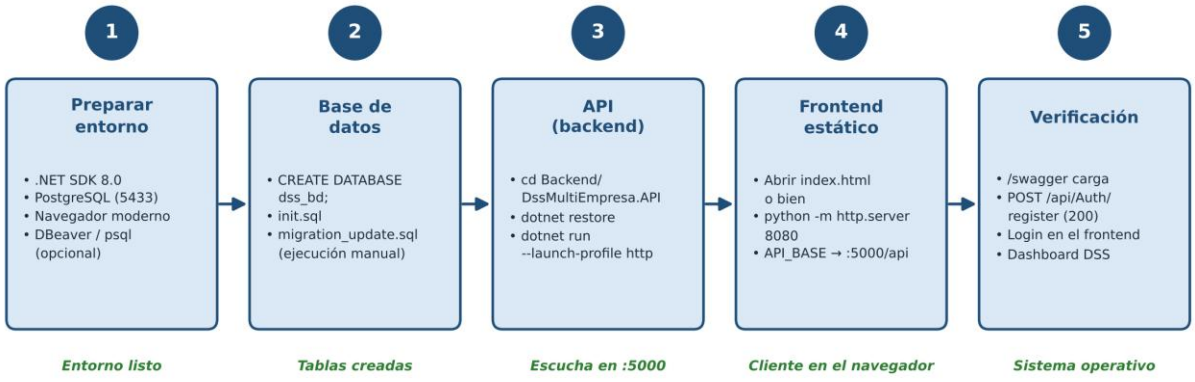


Nota. Elaboración propia.

Adicionalmente, la Figura 5 resume las fases del procedimiento y el resultado esperado de cada una de ellas.

Figura 5

Fases del Procedimiento de Despliegue Local



Nota. Elaboración propia.

Inicialización de la Base de Datos y Scripts SQL

1. Verificar que el servicio PostgreSQL se encuentre activo y escuchando en el puerto 5433.

2. Crear la base de datos vacía desde la consola SQL o pgAdmin mediante la instrucción:

CREATE DATABASE dss_bd;

3. Cargar secuencialmente los scripts del directorio Database/ utilizando la utilidad psql:

psql -h localhost -p 5433 -U postgres -d dss_bd -f Database/init.sql

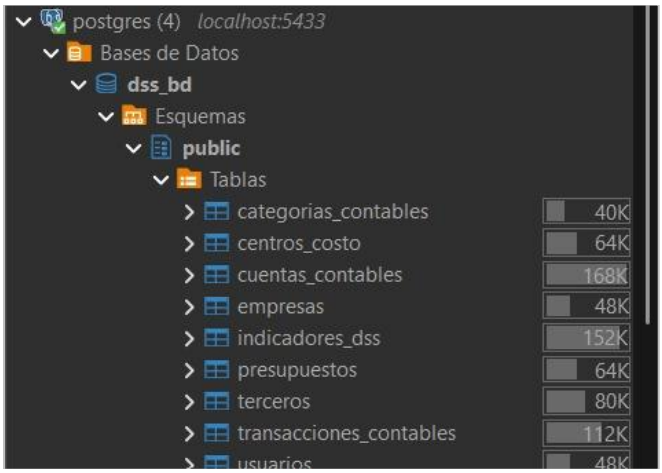
psql -h localhost -p 5433 -U postgres -d dss_bd -f Database/migration_update.sql

El script init.sql genera las tablas de la estructura básica, los índices relacionales e inserta el usuario por defecto demo. Por su parte, migration_update.sql es un script idempotente (utiliza cláusulas IF NOT EXISTS) que extiende las tablas contables y de empresas, garantizando su ejecución repetida sin provocar errores de duplicidad.

Tras la ejecución de los scripts, la base de datos dss_bd debe mostrar las nueve tablas del sistema, como se evidencia en la Figura 6.

Figura 6

Tablas Creadas en la Base de Datos dss_bd



Nota. Captura de pantalla de DBeaver conectado a localhost:5433. Se listan las tablas del esquema public; la última, usuarios, aparece parcialmente visible.

Compilación y Ejecución del Backend (.NET API)

Para iniciar el servidor de servicios API, navegue a la carpeta del proyecto backend y ejecute los comandos de restauración y lanzamiento de .NET CLI:

cd Backend/DssMultiEmpresa.API

dotnet restore

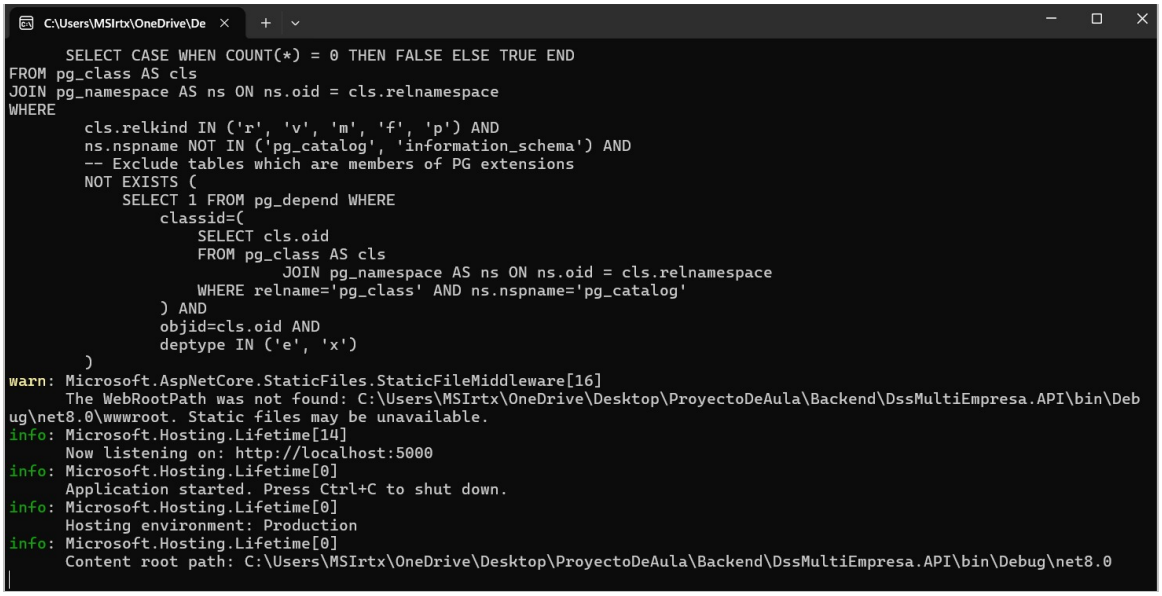
dotnet run --launch-profile http

Una vez iniciada la consola de comandos, la API quedará escuchando peticiones en el puerto 5000 y la interfaz interactiva de pruebas Swagger UI estará disponible en la ruta /swagger.

La Figura 7 muestra la salida de la consola durante el arranque de la API, donde se confirma que el servidor escucha en http://localhost:5000. En esa misma salida aparece una advertencia sobre la ausencia de la carpeta wwwroot, lo cual corrobora que la API no sirve archivos estáticos y que el frontend debe publicarse de forma independiente.

Figura 7

Salida de Consola durante el Arranque de la API

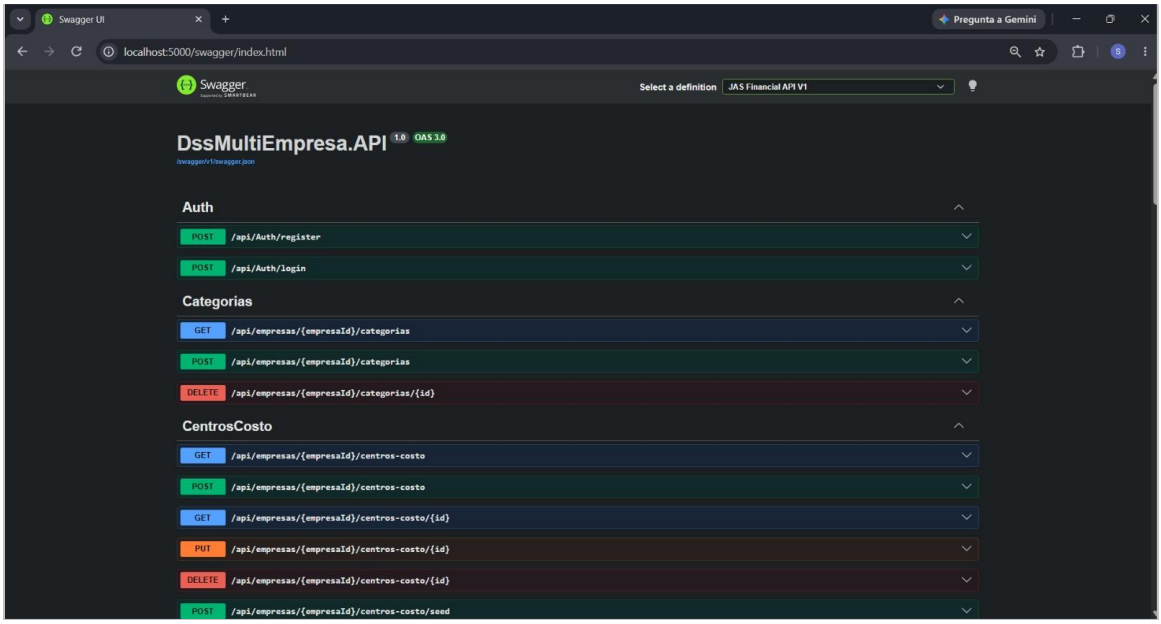


Nota. Captura de la consola de comandos. En esta ejecución se utilizó el binario compilado en Backend/DssMultiEmpresa.API/bin/Debug/net8.0, por lo que el entorno de hospedaje se reporta como Production.

Por su parte, la Figura 8 presenta la interfaz Swagger UI disponible en la ruta /swagger, con los grupos de endpoints expuestos por la API.

Figura 8

Interfaz Swagger UI de la API DssMultiEmpresa



Nota. Captura de pantalla de http://localhost:5000/swagger/index.html.

Despliegue y Pruebas del Frontend Estático

Dado que el proyecto frontend consiste en archivos estáticos desvinculados del servidor web de la API (a pesar de la llamada a UseStaticFiles() en Program.cs, el directorio wwwroot no existe en el proyecto), la interfaz se puede lanzar mediante dos modalidades:

• Opción A (Directa): Abrir el archivo Frontend/index.html directamente en el navegador web.

• Opción B (Servidor Estático Local): Ejecutar un servidor web liviano, por ejemplo mediante Python HTTP Server:

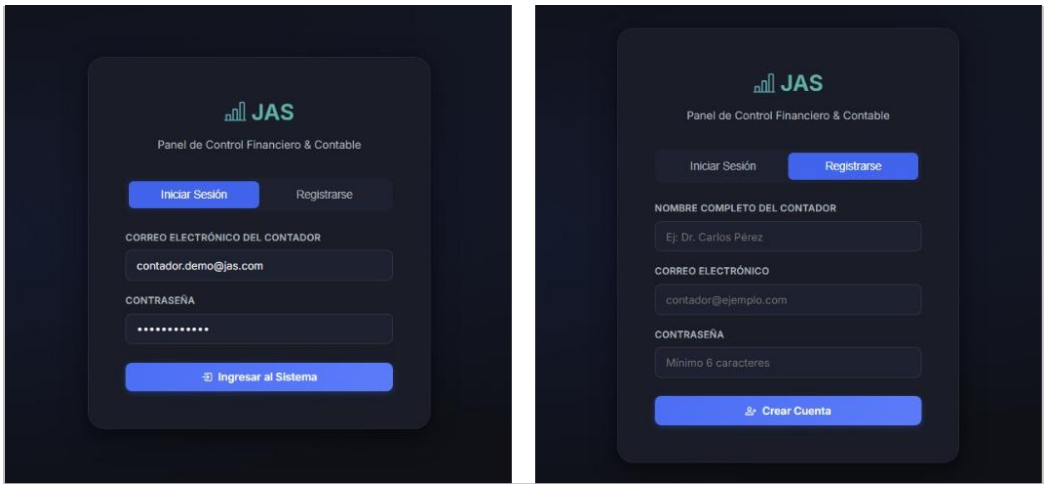
cd Frontend && python -m http.server 8080

En ambos escenarios, el cliente conectará exitosamente con el backend gracias a la permisividad de la política CORS configurada.

La Figura 9 muestra las pantallas de inicio de sesión y de registro del frontend, y la Figura 10 el panel principal (Dashboard DSS) una vez autenticado el usuario, lo que evidencia la comunicación correcta entre el cliente estático y la API.

Figura 9

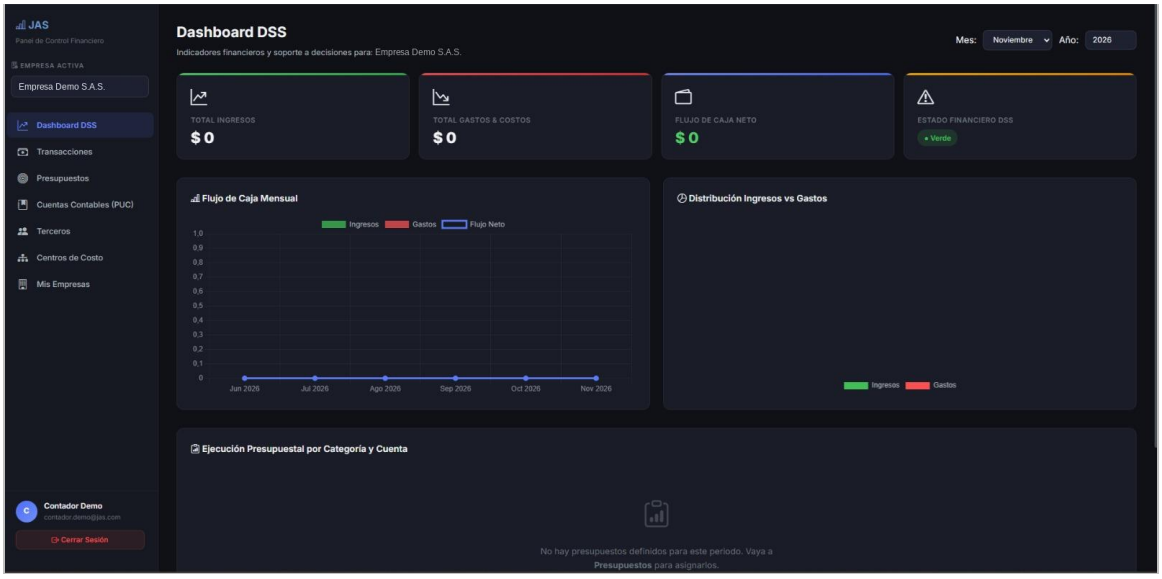
Pantallas de Inicio de Sesión y Registro del Frontend



Nota. Captura de pantalla del frontend en ejecución. A la izquierda, el formulario de inicio de sesión; a la derecha, el de registro de usuarios.

Figura 10

Panel Principal (Dashboard DSS) del Sistema JAS



Nota. Captura de pantalla del tablero tras iniciar sesión. Los indicadores aparecen en cero porque la empresa de prueba aún no registra movimientos. El nombre de la empresa fue sustituido por uno ficticio para proteger datos personales.

Análisis Diagnóstico del Proceso de Migración Automática

Durante la fase de inicialización del servidor backend, la clase Program.cs ejecuta dentro de un bloque try/catch un proceso automatizado de verificación de la base de datos:

1. Se invoca db.Database.EnsureCreated() para construir las tablas a partir del modelo de EF Core si la base no existe.
2. Se intenta leer el archivo SQL de actualización mediante la instrucción de ruta relativa AppContext.BaseDirectory/../../../../Database/migration_update.sql para ejecutarlo mediante ExecuteSqlRaw.

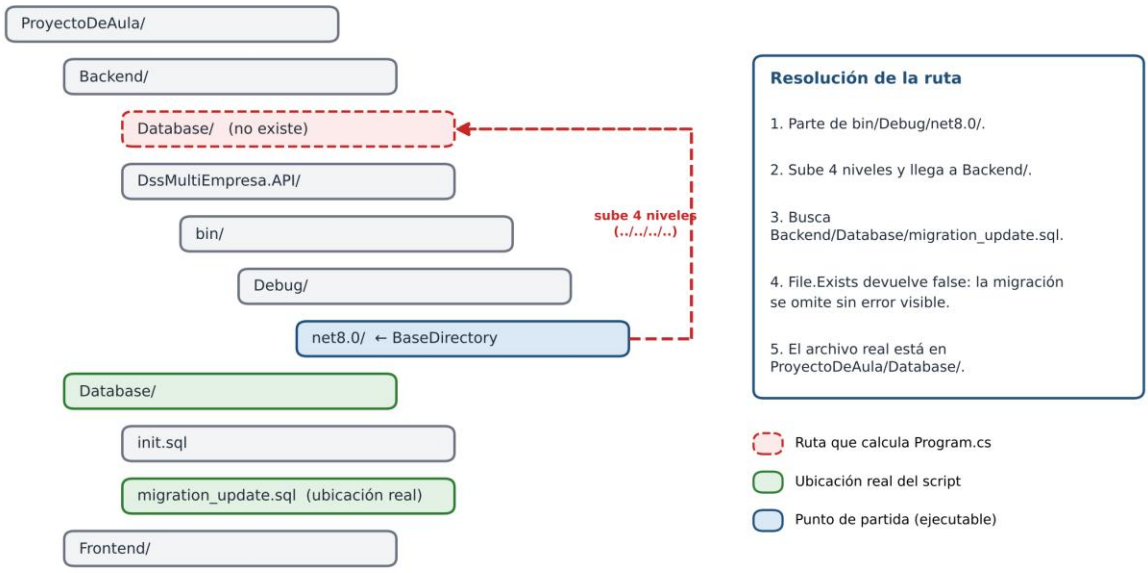
Discrepancia de Rutas Relativas en Program.cs

Un análisis detallado de la estructura de compilación revela una imperfección técnica en el cálculo de la ruta relativa: AppContext.BaseDirectory resuelve a la carpeta de salida ejecutable Backend/DssMultiEmpresa.API/bin/Debug/net8.0/. Al ascender cuatro niveles jerárquicos (../../../../), la ruta resultante se ubica en Backend/Database/migration_update.sql. Sin embargo, en la estructura física del repositorio entregado, la carpeta Database/ se encuentra ubicada un nivel superior (en la raíz ProyectoDeAula/Database/).

La Figura 11 representa gráficamente este cálculo, contrastando el destino que resuelve Program.cs con la ubicación real del script.

Figura 11

Resolución de la Ruta de migration_update.sql en Program.cs



Nota. Elaboración propia a partir de la estructura del repositorio y del código de Program.cs.

En consecuencia, la validación File.Exists evalúa como falsa, provocando que la migración automática se omita en silencio sin detener la ejecución de la API (debido a la captura de excepciones mediante advertencias en el log). Por esta razón, resulta imperativo realizar la carga manual de los scripts SQL especificada en la sección de despliegue.

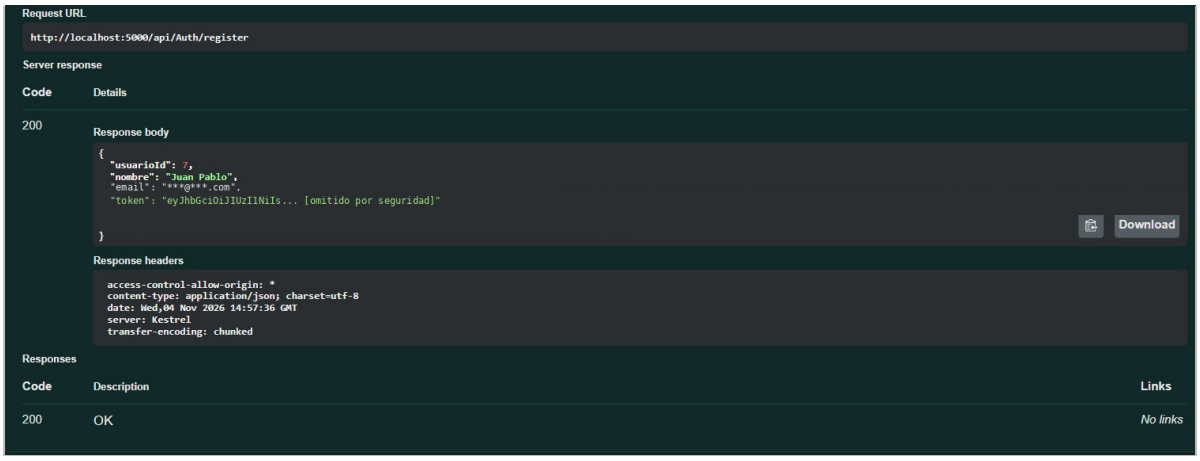
Gestión de Usuarios y Autenticación JWT

El script init.sql incluye la inserción del usuario de pruebas almacenando una contraseña cifrada mediante hash. Dado que la clave en texto plano del usuario de demostración no se encuentra documentada en el código fuente, la vía idónea para realizar pruebas iniciales consiste en crear una nueva cuenta de usuario a través del endpoint público /api/auth/register o directamente desde el formulario de registro del frontend.

La Figura 12 evidencia el registro exitoso de un usuario de prueba mediante Swagger: el servidor responde con código 200 y devuelve el identificador del usuario junto con un token JWT.

Figura 12

Registro de un Usuario de Prueba mediante el Endpoint /api/Auth/register



Nota. Captura de pantalla de Swagger UI. Por seguridad, el token JWT y el correo electrónico fueron ocultados y se omitió el cuerpo de la solicitud, que incluía la contraseña.

Recomendaciones para el Entorno de Producción

Para la transición de la solución JAS hacia un entorno de producción seguro, escalable y mantenible, se recomienda implementar las siguientes mejoras estructurales:

1. Corrección de Rutas y Migraciones EF Core: Sustituir la lectura directa de scripts SQL en Program.cs por el mecanismo nativo de migraciones de Entity Framework Core (EF Core Migrations), incorporando una carpeta de código de migraciones gestionada mediante dotnet ef migrations add.

2. Contenerización con Docker: Crear un archivo Dockerfile para el backend API y un orquestador docker-compose.yml que despliegue conjuntamente la API y la base de datos PostgreSQL, garantizando la portabilidad del entorno.

3. Tubería CI/CD: Configurar un pipeline automatizado de integración y despliegue continuo para pruebas unitarias, análisis estático de código y publicación automática de imágenes.
4. Seguridad y Gestión de Secretos: Externalizar las claves secretas de JWT y las credenciales de PostgreSQL fuera del archivo appsettings.json, utilizando variables de entorno o gestores de secretos (Azure Key Vault, HashiCorp Vault).
5. Restricción de CORS y TLS: Reemplazar el comodín AllowAnyOrigin en las políticas CORS por los dominios exactos del cliente en producción y forzar el uso de certificados de cifrado HTTPS/TLS.

Conclusiones

El sistema JAS – Panel de Control Financiero y Contable ofrece una arquitectura limpia y moderna basada en la separación de responsabilidades entre un backend REST en .NET 8, una base de datos relacional PostgreSQL y una interfaz web estática. El análisis técnico realizado permitió verificar la viabilidad completa de su despliegue local manual de forma totalmente autónoma e independiente, identificando con precisión la solución al comportamiento de las migraciones automáticas en Program.cs y estableciendo la hoja de ruta para su posterior adaptación a estándares corporativos en contenedores Docker.

Referencias

Microsoft. (2024). *Documentación de ASP.NET Core 8 Web API y Entity Framework Core.* Microsoft Learn.

PostgreSQL Global Development Group. (2024). *PostgreSQL 16.0 Documentation: Database Administration and SQL Language.* PostgreSQL Documentation.

The Open Web Application Security Project. (2023). *OWASP Top 10 API Security Risks Solution Guide.* OWASP Foundation.

IETF Cose Working Group. (2015). *JSON Web Token (JWT) Architecture and Security Best Practices.* Internet Engineering Task Force.